

Yesterday 8:38 PM

Hey, I want help with my assessment.

Here's the github link..

Lets do it together .. guide me on how do i do this and explain each step by step.

Absolutely 👍 Send me the **GitHub** link and, if you have it, the **assessment instructions/question** as well.

Once you send it, we'll do it **step by step together**:

1. **Understand the assessment** — what exactly is being asked.
2. **Explore the GitHub repo** — understand the existing code and structure.
3. **Set up the project locally** — I'll guide you through installation and running it.
4. **Break the assessment into small tasks** — so you know exactly what to do.
5. **Implement each part together** — I'll explain *why* we're doing each change, not just give you code.
6. **Run and test it** — we'll troubleshoot errors from your terminal as they come up.
7. **Review the final solution** — make sure it satisfies the assessment requirements.
8. **Prepare your submission/demo**, if the assessment requires one.

Send me the **GitHub URL** first, and I'll start by examining the repository and explaining what we're working with.

🔗 <https://github.com/cs409-fa25/mp2>

Yes — we can do it together step by step.

I checked the URL you sent. The exact `cs409-fa25/mp2` repository isn't accessible through my web fetch right now (it returns 404), so I don't want to guess what the assessment requires. (Invalid URL)

What I need from you

Please do **one** of these:

- **Option 1:** Open the GitHub repo and paste the contents of its `README.md` here.
- **Option 2:** Upload the assessment/MP2 PDF or instructions here.
- **Option 3:** If the repo is private, make it accessible or upload the repository files.

Once I have the actual assignment, we'll proceed like this:

Step 1 → Understand the requirements

I'll explain every requirement in very simple terms.

Step 2 → Set up the repo

I'll give you the exact commands to clone/install/run it.

Step 3 → Understand the existing code

We'll go through the important files and understand what each one does.

Step 4 → Implement the assessment

We'll work through **one requirement at a time**. I'll explain the logic first, then we'll write the code.

Step 5 → Test locally

You'll run it and send me your output/errors. We'll fix them together.

Step 6 → Final check

We'll compare your implementation against the assignment requirements and make sure you haven't missed anything.

Since this is an assessment, I'll focus on **teaching you what the code is doing and why**, rather than just dumping a finished solution.

Send me the README/assessment instructions, and we'll start with Step 1.

Today 3:08 AM



mp2-main.zip
File

Worked for 9s

Analyzed

Analyzed

Perfect. I have the actual MP now. The deadline in the README is **October 6, 2026 at 11:59 PM CT**, and the assignment explicitly allows LLM use as long as you submit the chat logs and answer the related survey questions.

We're going to build this in a way that you understand every part.

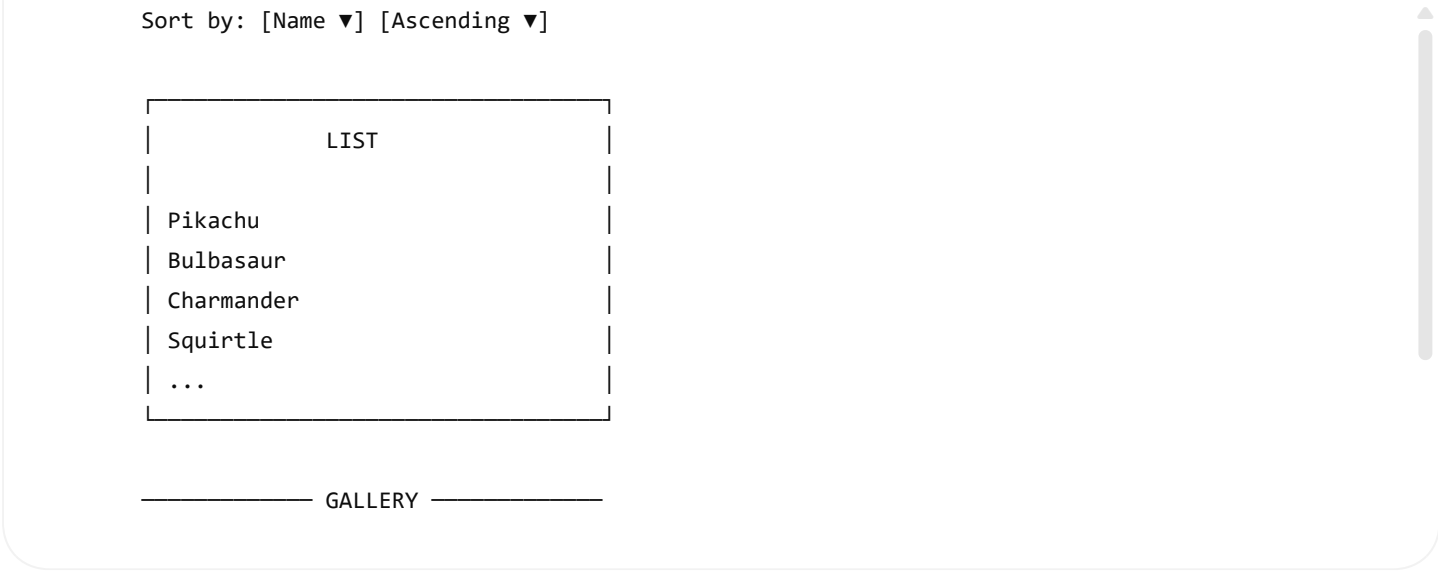
1. What we're building

We'll use **PokéAPI** and make an app roughly like this:

</> Plain text

Pokémon Explorer

| Search Pokémon: [pika...] |



This maps directly onto the rubric.

Rubric → our implementation

Requirement	Our implementation
List view	Pokémon list
Search	Search Pokémon by name
Filter as you type	React state + <code>.filter()</code>
Sort property #1	Name
Sort property #2	ID
Ascending	A → Z / low → high
Descending	Z → A / high → low
Gallery	Pokémon images
Gallery filter	Pokémon type
List → Details	React Router
Gallery → Details	React Router
Details	Image, type, stats, height, weight, etc.
Previous	Previous Pokémon

Requirement	Our implementation
Next	Next Pokémon
React Router	react-router-dom
Axios	All API requests
TypeScript	.tsx / .ts
Design	We'll make it look polished

2. First thing: get your project running

The ZIP you uploaded contains only:

</> Plain text

```
mp2-main/  
├─ README.md  
└─ .github/  
    └─ workflows/  
        └─ deploy.yml
```

That's intentional — you are supposed to scaffold the React project yourself.

Step 1 — create your own repository

From the class repository, use:

[Use this template](#) → [Create a new repository](#)

Name it:

</> Plain text

```
mp2
```

Make it **public**.

Then clone your repository.

For example:

</> Bash

```
git clone git@github.com:YOUR_USERNAME/mp2.git
cd mp2
```

If you are using HTTPS instead:

</> Bash

```
git clone https://github.com/YOUR_USERNAME/mp2.git
cd mp2
```

3. Create the React + TypeScript project

Inside the `mp2` directory:

</> Bash

```
npm create vite@latest . -- --template react-ts
```

Vite may ask what to do because the repository already contains:

</> Plain text

```
README.md
.github/
```

Choose:

</> Plain text

```
Ignore files and continue
```

Do NOT choose Remove existing files.

That's important because:

</> Plain text

```
.github/workflows/deploy.yml
```

is what will eventually deploy your website to GitHub Pages.

Then:

```
</> Bash
```

```
npm install
```

4. Install the two required libraries

Your assignment specifically requires:

- React Router
- Axios

Install them:

```
</> Bash
```

```
npm install react-router-dom axios
```

So eventually your important dependencies will include:

```
</> Plain text
```

```
react  
react-dom  
react-router-dom  
axios
```

5. Run the starter app

Run:

```
</> Bash
```

```
npm run dev
```

You should see something like:

```
</> Plain text
```

```
Local: http://localhost:5173/
```

Open:

</> Plain text

http://localhost:5173/

You should see the default Vite React page.

Don't modify anything yet.

First let's make sure your environment works.

6. Understand the architecture before coding

We're going to organize the project something like this:

</> Plain text

```
src/
|
├── components/
|   ├── SearchBar.tsx
|   ├── SortControls.tsx
|   ├── PokemonList.tsx
|   ├── PokemonCard.tsx
|   ├── PokemonGallery.tsx
|   └── FilterControls.tsx
|
├── pages/
|   ├── Home.tsx
|   └── PokemonDetails.tsx
|
├── services/
|   └── pokemonApi.ts
|
├── types/
|   └── pokemon.ts
|
```

Don't worry if that looks like a lot.

Each folder has a simple purpose.

components/

Reusable UI pieces.

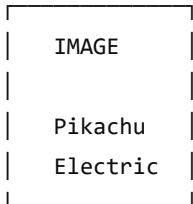
For example:

</> Plain text

PokemonCard

will display:

</> Plain text



pages/

Actual screens/routes.

We'll have:

</> Plain text

/

for the main application.

And:

</> Plain text

/pokemon/:id

for details.

For example:

</> Plain text

/pokemon/25

means:

Show me Pokémon whose ID is 25.

services/

This is where Axios/API code lives.

Instead of putting API requests everywhere, we'll have something like:

</> Plain text

```
pokemonApi.ts
```

that handles communication with PokéAPI.

Conceptually:

</> Plain text

```
React app
  ↓
pokemonApi.ts
  ↓
Axios
  ↓
PokéAPI
  ↓
JSON data
  ↓
React app
```

types/

TypeScript definitions.

For example, we'll eventually tell TypeScript that a Pokémon has:

</> Plain text

```
id
name
height
weight
```

types
sprites
stats

This is important because the assignment specifically requires **TypeScript**.

7. How the data flow will work

This is the most important concept to understand before we start coding.

Suppose the user opens the application.

React makes a request:

⌄ Plain text

```
GET https://pokeapi.co/api/v2/pokemon?limit=151
```

The API gives us Pokémon information.

Then our React application stores it in state:

⌄ Plain text

```
API  
↓  
Axios  
↓  
useEffect()  
↓  
useState()  
↓  
UI
```

For example:

⌄ Plain text

```
PokéAPI  
↓  
151 Pokémon  
↓  
pokemonData  
↓  
Search
```

↓
Filter
↓
Sort
↓
Display

The nice thing is that **searching and sorting can happen on the client side**, exactly as the assignment allows.

8. Why I'm suggesting the first 151 Pokémon

Instead of loading thousands of Pokémon, we'll initially work with the original 151.

That gives us enough data for:

- search
- sorting
- gallery
- type filtering
- details
- previous/next

without making the application unnecessarily complicated.

For example:

</> Plain text

```
1  Bulbasaur
2  Ivysaur
3  Venusaur
...
25 Pikachu
...
150 Mewtwo
151 Mew
```

Then:

</> Plain text

Search: "pika"

becomes:

</> Plain text

Pikachu

And:

</> Plain text

Type: Fire

might show:

</> Plain text

Charmander

Charmeleon

Charizard

Vulpix

...

9. One important design decision

For the **list view**, we'll probably show something like:

</> Plain text

Search Pokémon

[pika_____]

Sort by:

[Name ▼]

Order:

[Ascending ▼]

#25 Pikachu

#26 Raichu

...

The two sortable properties will be:

Property 1 — Name

</> Plain text

Bulbasaur
Charmander
Pikachu
Squirtle

Property 2 — ID

</> Plain text

1
2
3
...
25
...
151

And each can be:

</> Plain text

Ascending
Descending

So we're clearly satisfying:

Can you sort by at least 2 properties?

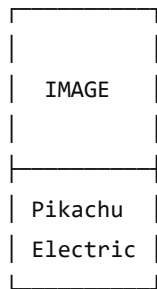
and:

Can the properties be sorted in Ascending and Descending order?

10. Gallery

The gallery will use the Pokémon artwork:

</> Plain text



Above it:

`</>` Plain text

Filter by Type

[All] [Normal] [Fire] [Water]
[Electric] [Grass] [Poison] ...

When the user clicks:

`</>` Plain text

Fire

we filter the gallery.

When they click:

`</>` Plain text

Water

we show Water Pokémon.

This directly addresses the gallery-filter requirement.

11. Details route

This is particularly important because the assignment explicitly says the details page must have a **specific URL**.

We'll use React Router.

For example:

</> Plain text

```
/pokemon/25
```

The URL itself identifies the Pokémon.

So if you copy:

</> Plain text

```
https://yourusername.github.io/mp2/pokemon/25
```

someone can directly visit that route.

The page will show something like:

</> Plain text

Pikachu

[BIG IMAGE]

Type: Electric

Height: 4

Weight: 60

Base Stats

HP	35
----	----

Attack	55
--------	----

Defense	40
---------	----

Speed	90
-------	----

[← PREVIOUS] [NEXT →]

12. Previous / Next

This part is actually quite straightforward once the data structure is set up.

If we're viewing:

</> Plain text

```
/pokemon/25
```

Previous is:

```
</> Plain text
```

```
24
```

Next is:

```
</> Plain text
```

```
26
```

So clicking:

```
</> Plain text
```

```
NEXT →
```

changes the route to:

```
</> Plain text
```

```
/pokemon/26
```

and React loads that Pokémon.

Likewise:

```
</> Plain text
```

```
← PREVIOUS
```

goes to:

```
</> Plain text
```

```
/pokemon/24
```

This will earn the **10 points** for the Previous/Next requirement.

13. Our implementation order

Don't try to build all of this at once.

We'll do it in this order:

Phase 1 — Setup

</> Plain text

Vite
React
TypeScript
Axios
React Router

Phase 2 — API

Get Pokémon data working.

Phase 3 — TypeScript types

Define what a Pokémon looks like.

Phase 4 — List

Display Pokémon.

Phase 5 — Search

Make:

</> Plain text

[pika]

filter the list as the user types.

Phase 6 — Sorting

Implement:

</> Plain text

Name
ID

and:

Plain text

Ascending
Descending

Phase 7 — Gallery

Display images.

Phase 8 — Gallery filtering

Filter by Pokémon type.

Phase 9 — React Router

Create:

Plain text

/pokemon/:id

Phase 10 — Details

Display full Pokémon information.

Phase 11 — Previous / Next

Implement navigation.

Phase 12 — Styling

Make the application look good.

Phase 13 — GitHub Pages

Configure:

Plain text

vite.config.ts

and:

Plain text

BrowserRouter

Phase 14 — Deployment

Push to GitHub → GitHub Actions → GitHub Pages.

Phase 15 — Demo video

We'll make sure your 3-minute demo explicitly shows every rubric item.

Your first task

Don't write any application code yet.

Go into your new `mp2` repository and run:

```
</> Bash
```

```
npm create vite@latest . -- --template react-ts
```

then:

```
</> Bash
```

```
npm install
```

```
npm install react-router-dom axios
```

and finally:

```
</> Bash
```

```
npm run dev
```

Stop there.

Once you see the Vite page at `localhost:5173`, tell me **"it's running"** (or send me the error/screenshot if something goes wrong).

Then we'll build **Phase 2: our first Axios request to PokéAPI**, and I'll explain exactly what every line of that code means. </>

done

Perfect 👍 Let's move to **Phase 2: connect the app to PokéAPI**.

Step 1 — Create the API service

Inside `src`, create a folder:

</> Plain text

```
src/  
└─ services/
```

Inside `services`, create:

</> Plain text

```
pokemonApi.ts
```

Put this in it:

</> TypeScript

```
import axios from "axios";  
  
const API_URL = "https://pokeapi.co/api/v2";  
  
export const getPokemonList = async () => {  
  const response = await axios.get(`${API_URL}/pokemon?limit=151`);  
  return response.data;  
};
```

What this does

Think of it as a little function whose job is:

"Go to PokéAPI and bring me the first 151 Pokémon."

This line:

</> TypeScript

```
axios.get(...)
```

makes the HTTP request.

The API returns JSON.

Then:

</> TypeScript

```
response.data
```

extracts the actual data from the response.

So:

</> Plain text

```
React
  ↓
getPokemonList()
  ↓
Axios
  ↓
PokéAPI
  ↓
JSON
  ↓
response.data
```

Step 2 — Test the API

Now let's actually call it from React.

Open:

</> Plain text

```
src/App.tsx
```

Replace its contents temporarily with:

</> TypeScript

```
import { useEffect, useState } from "react";
import { getPokemonList } from "../services/pokemonApi";

function App() {
  const [pokemon, setPokemon] = useState<any[]>([]);

  useEffect(() => {
    const loadPokemon = async () => {
      const data = await getPokemonList();
      setPokemon(data.results);
    };

    loadPokemon();
  }, []);
```

```
return (  
  <div>  
    <h1>Pokémon Explorer</h1>  
  
    <p>Total Pokémon: {pokemon.length}</p>  
  </div>  
)
```

Step 3 — Understand the important part

This:

</> TypeScript

```
const [pokemon, setPokemon] = useState<any[]>([]);
```

creates React state.

Initially:

</> Plain text

```
pokemon = []
```

After the API request:

</> Plain text

```
pokemon = [  
  { name: "bulbasaur", url: "..."},  
  { name: "ivysaur", url: "..."},  
  ...  
]
```

Then this:

</> TypeScript

```
useEffect(() => {
```

means:

Run this code when the component loads.

Inside it:

</> TypeScript

```
const data = await getPokemonList();
```

calls our API function.

Then:

</> TypeScript

```
setPokemon(data.results);
```

puts the API data into React state.

Finally:

</> TypeScript

```
pokemon.map((p) => (  
  <li key={p.name}>{p.name}</li>  
))
```

loops through the Pokémon and displays them.

So the flow is:

</> Plain text

```
App loads  
↓  
useEffect()  
↓  
getPokemonList()  
↓  
Axios  
↓  
PokéAPI  
↓  
data.results  
↓  
setPokemon(...)  
↓  
React re-renders
```

↓
Pokémon appear

Step 4 — Run it

Save the files and look at:

⌄ Plain text

`http://localhost:5173`

You should see something like:

⌄ Plain text

Pokémon Explorer

Total Pokémon: 151

- bulbasaur
- ivysaur
- venusaur
- charmander
- charmeleon
- ...
- mew

Don't worry about the `any[]` yet. We're deliberately using it temporarily so you can first understand the API → React flow. In the next step, we'll replace it with proper **TypeScript interfaces**, because your assessment requires TypeScript.

Once you see the 151 Pokémon, tell me **"I see them"** and we'll build the proper TypeScript data model.

done

Great! 👍 Now let's do **Phase 3: TypeScript types**.

Right now we used:

⌄ TypeScript

`useState<any[]>([])`

That works, but `any` basically tells TypeScript:

"Don't check this data."

For the assessment, we want proper TypeScript.

Step 1 — Create a `types` folder

Inside `src` :

</> Plain text

```
src/  
├─ services/  
│   └─ pokemonApi.ts  
├─ types/  
│   └─ pokemon.ts  
├─ App.tsx  
└─ main.tsx
```

Create:

</> Plain text

```
src/types/pokemon.ts
```

Put this in it:

</> TypeScript

```
export interface PokemonListItem {  
  name: string;  
  url: string;  
}  
  
export interface PokemonListResponse {  
  results: PokemonListItem[];  
  count: number;  
  next: string | null;  
  previous: string | null;  
}
```

What are these?

The API gives us something roughly like:

</> JSON

```
{
  "count": 1350,
  "next": "...",
  "previous": null,
  "results": [
    {
      "name": "bulbasaur",
      "url": "https://pokeapi.co/api/v2/pokemon/1/"
    }
  ]
}
```

So we're telling TypeScript:

</> Plain text

PokemonListItem

↓

name = string

url = string

and:

</> Plain text

PokemonListResponse

↓

count

next

previous

results

Step 2 — Update `pokemonApi.ts`

Change it to:

</> TypeScript

```
import axios from "axios";
```

```
import type { PokemonListResponse } from "../types/pokemon";
```

```
const API_URL = "https://pokeapi.co/api/v2";
```

```
export const getPokemonList = async (): Promise<PokemonListResponse> => {
```

```
  const response = await axios.get<PokemonListResponse>(
```

```
    `${API_URL}/pokemon?limit=151`
```

```
);  
  
    return response.data;  
};
```

Notice this:

</> TypeScript

```
axios.get<PokemonListResponse>
```

We're telling TypeScript:

"I expect the API response to have the structure described by `PokemonListResponse` ."

And:

</> TypeScript

```
Promise<PokemonListResponse>
```

means:

"This async function will eventually return a `PokemonListResponse` ."

Step 3 — Fix `App.tsx`

Now change:

</> TypeScript

```
const [pokemon, setPokemon] = useState<any[]>([]);
```

to:

</> TypeScript

```
import type { PokemonListItem } from "../types/pokemon";
```

and:

</> TypeScript

```
const [pokemon, setPokemon] = useState<PokemonListItem[]>([]);
```

Your complete `App.tsx` should now be:

`</>` TypeScript

```
import { useEffect, useState } from "react";
import { getPokemonList } from "../services/pokemonApi";
import type { PokemonListItem } from "../types/pokemon";

function App() {
  const [pokemon, setPokemon] = useState<PokemonListItem[]>([]);

  useEffect(() => {
    const loadPokemon = async () => {
      const data = await getPokemonList();
      setPokemon(data.results);
    };

    loadPokemon();
  }, []);

  return (
    <div>
      <h1>Pokémon Explorer</h1>
```

Why we're doing this

This is a very important concept for your assessment.

Previously:

`</>` Plain text

API → any → React

Now:

`</>` Plain text

API → TypeScript type → React

So if you accidentally write:

</> TypeScript

p.age

TypeScript can tell you:

Property age does not exist on PokemonListItem .

That's one of the main benefits of TypeScript.

One more thing

Our current API data only contains:

</> Plain text

name
url

But our final app needs:

</> Plain text

image
types
stats
height
weight

So we're going to create another TypeScript interface for the **full Pokémon details**.

We'll need that when we build the details page.

For now, make these two changes and make sure the app still shows the 151 Pokémon.

Once it works, we'll move to **Phase 4: fetching the actual Pokémon details and images**.

done

Awesome. Now we're ready for **Phase 4: fetch the full Pokémon data**.

Right now our API only gives us:

</> Plain text

```
name
url
```

But for our app we need:

</> Plain text

```
name
id
image
types
height
weight
stats
```

We'll get that from each Pokémon's individual API endpoint.

Step 1 — Understand the API structure

For example, our list API gives:

</> Plain text

```
bulbasaur
https://pokeapi.co/api/v2/pokemon/1/
```

If we request that URL, we get the full Bulbasaur data.

Similarly:

</> Plain text

```
pikachu
https://pokeapi.co/api/v2/pokemon/25/
```

gives us Pikachu's full data.

So our application will eventually do:

</> Plain text

```
List API
  ↓
151 Pokémon URLs
  ↓
Individual API requests
```



Full Pokémon information



Store in React

Step 1 — Add the full Pokémon type

Open:

 Plain text

src/types/pokemon.ts

Keep your existing interfaces and **add** this:

 TypeScript

```
export interface PokemonType {
  slot: number;
  type: {
    name: string;
    url: string;
  };
}

export interface PokemonStat {
  base_stat: number;
  effort: number;
  stat: {
    name: string;
    url: string;
  };
}

export interface PokemonDetails {
  id: number;
  name: string;
```

So the entire file should now look like:

 TypeScript

```
export interface PokemonListItem {
  name: string;
  url: string;
}
```

```
export interface PokemonListResponse {
  results: PokemonListItem[];
  count: number;
  next: string | null;
  previous: string | null;
}

export interface PokemonType {
  slot: number;
  type: {
    name: string;
    url: string;
  };
}
```

Step 2 — Add a function to fetch one Pokémon

Open:

</> Plain text

src/services/pokemonApi.ts

Add this import:

</> TypeScript

```
import type {
  PokemonListResponse,
  PokemonDetails,
} from "../types/pokemon";
```

Then add this function underneath `getPokemonList` :

</> TypeScript

```
export const getPokemonDetails = async (
  url: string
): Promise<PokemonDetails> => {
  const response = await axios.get<PokemonDetails>(url);

  return response.data;
};
```

Your file should now be:

</> TypeScript

```
import axios from "axios";
import type {
  PokemonListResponse,
  PokemonDetails,
} from "../types/pokemon";

const API_URL = "https://pokeapi.co/api/v2";

export const getPokemonList = async (): Promise<PokemonListResponse> => {
  const response = await axios.get<PokemonListResponse>(
    `${API_URL}/pokemon?limit=151`
  );

  return response.data;
};

export const getPokemonDetails = async (
  url: string
): Promise<PokemonDetails> => {
  const response = await axios.get<PokemonDetails>(url);
```

Step 3 — Let's test ONE Pokémon

Before making 151 requests, let's test with Pikachu.

Temporarily change your `App.tsx` to:

</> TypeScript

```
import { useEffect, useState } from "react";
import { getPokemonDetails } from "../services/pokemonApi";
import type { PokemonDetails } from "../types/pokemon";

function App() {
  const [pokemon, setPokemon] = useState<PokemonDetails | null>(null);

  useEffect(() => {
    const loadPokemon = async () => {
      const data = await getPokemonDetails(
        "https://pokeapi.co/api/v2/pokemon/25/"
      );

      setPokemon(data);
    };
  });
}
```

```
loadPokemon();  
}, []);  
  
if (pokemon) {
```

What you should see

Your browser should now show something roughly like:

</> Plain text

Pikachu

[Pikachu image]

ID: 25

Height: 4

Weight: 60

Types

- electric

Stats

- hp: 35
- attack: 55
- defense: 40
- special-attack: 50

 This is the foundation of our details page.

Notice how much easier the final details page will be now.

Why we're testing one Pokémon first

This is an important development habit.

We could immediately make:

</> Plain text

151 API requests

But if something goes wrong, we'd have no idea whether the problem is:

- API request
- TypeScript
- image URL
- data structure
- React state
- rendering

Instead we're doing:

⌄ Plain text

```
1 Pokémon
  ↓
verify everything
  ↓
151 Pokémon
```

This makes debugging much easier.

Your task now

Run the app and confirm that you can see **Pikachu's image, ID, type, and stats**.

Once that's working, we'll move to the really interesting part: **loading all 151 Pokémon efficiently and building the list/search functionality**.

done

Excellent. Now we're going to build the **actual list view**. This is where the assessment starts coming together.

We'll do it in small pieces:

1. Fetch all 151 Pokémon details.
2. Store them in state.
3. Display them as a list.
4. Add the search box.
5. Make search work **as the user types**.

We won't do sorting yet.

Step 5 — Fetch all 151 Pokémon

First, change `App.tsx` back from the temporary Pikachu test.

We'll fetch the list and then fetch each Pokémon's details.

Replace `App.tsx` with:

`</>` TypeScript

```
import { useEffect, useState } from "react";
import {
  getPokemonDetails,
  getPokemonList,
} from "../services/pokemonApi";
import type {
  PokemonDetails,
  PokemonListItem,
} from "../types/pokemon";

function App() {
  const [pokemon, setPokemon] = useState<PokemonDetails[]>([]);

  useEffect(() => {
    const loadPokemon = async () => {
      const listData = await getPokemonList();

      const details = await Promise.all(
        listData.results.map((item: PokemonListItem) =>
          getPokemonDetails(item.url)
        )
      );
    };
  });
}
```

What is `Promise.all()` doing?

This is important.

We have:

`</>` Plain text

151 Pokémon URLs

Instead of doing:

`</>` Plain text

Pokémon 1
wait
Pokémon 2
wait
Pokémon 3

```
wait  
...
```

`Promise.all()` lets us request them together.

Conceptually:

`</>` Plain text

```
├─ Pokémon 1 API  
├─ Pokémon 2 API  
├─ Pokémon 3 API  
├─ Pokémon 4 API  
├─ ...  
└─ Pokémon 151 API  
    ↓  
  Promise.all()  
    ↓  
All results
```

Then:

`</>` TypeScript

```
setPokemon(details);
```

stores all of them in React state.

Step 6 — Add Search

Now we're going to introduce an important React concept:

State vs derived data

We have our complete dataset:

`</>` TypeScript

```
pokemon
```

We **don't** want to modify it when searching.

Instead we'll create another state:

```
</> TypeScript
```

```
const [search, setSearch] = useState("");
```

This stores whatever the user has typed.

For example:

```
</> Plain text
```

```
search = ""
```

Initially.

Then:

```
</> Plain text
```

```
search = "p"
```

Then:

```
</> Plain text
```

```
search = "pi"
```

Then:

```
</> Plain text
```

```
search = "pik"
```

Then:

```
</> Plain text
```

```
search = "pika"
```

Add this state

At the top of `App()` :

</> TypeScript

```
const [search, setSearch] = useState("");
```

So:

</> TypeScript

```
function App() {  
  const [pokemon, setPokemon] = useState<PokemonDetails[]>([]);  
  const [search, setSearch] = useState("");  
  
  ...  
}
```

Step 7 — Create filtered Pokémon

Below your `useEffect`, add:

</> TypeScript

```
const filteredPokemon = pokemon.filter((p) =>  
  p.name.toLowerCase().includes(search.toLowerCase())  
);
```

This is the actual search logic.

Suppose:

</> Plain text

```
search = "char"
```

We check:

</> Plain text

```
bulbasaur  
ivysaur  
charmander ← match  
charmeleon ← match  
charizard ← match  
squirtle
```

Result:

</> Plain text

```
charmander  
charmeleon  
charizard
```

Step 8 — Add the search box

Replace your return section with:

</> TypeScript

```
return (  
  <div>  
    <h1>Pokémon Explorer</h1>  
  
    <input  
      type="text"  
      placeholder="Search Pokémon..."  
      value={search}  
      onChange={(e) => setSearch(e.target.value)}  
    />  
  
    <p>  
      Showing {filteredPokemon.length} of {pokemon.length} Pokémon  
    </p>  
  
    <ul>  
      {filteredPokemon.map((p) => (  
        <li key={p.id}>  
          #{p.id} {p.name}  
        </li>  
      )  
    }  
  )  
)
```

The important React part

Look at:

</> TypeScript

```
value={search}
```

This means:

The input's value is controlled by React state.

Then:

</> TypeScript

```
onChange={(e) => setSearch(e.target.value)}
```

means:

Every time the user types something, update `search` .

For example:

</> Plain text

User types "p"

↓

`setSearch("p")`

↓

React re-renders

↓

`filteredPokemon` recalculates

↓

List changes

Then the user types:

</> Plain text

"pi"

and the process happens again.

That's exactly what the assignment means by:

filter as you type

Your app should now behave like this

Initially:

</> Plain text

Pokémon Explorer

[Search Pokémon...]

Showing 151 of 151 Pokémon

#1 bulbasaur

#2 ivysaur

#3 venusaur

...

Type:

</> Plain text

pika

and immediately:

</> Plain text

Pokémon Explorer

[pika]

Showing 1 of 151 Pokémon

#25 pikachu

Try:

</> Plain text

char

You should get:

</> Plain text

#4 charmander

#5 charmeleon

#6 charizard

Try:

</> Plain text

xyz

and you should get:

</> Plain text

Showing 0 of 151 Pokémon

One thing to notice

We're currently doing:

</> TypeScript

```
p.name.toLowerCase().includes(search.toLowerCase())
```

So search is **case-insensitive**.

That means:

</> Plain text

Pikachu
pikachu
PIKACHU
PiKaChU

all work.

That's a nice little usability improvement.

Do this now

Replace `App.tsx` with the version above, run it, and test:

</> Plain text

pika
char
xyz

If those work, **don't change anything else yet.**

Next we'll tackle **sorting by two properties + ascending/descending**, which is one of the major rubric requirements.

done

Perfect. 🚀 Now we tackle one of the main rubric requirements: **sorting by two properties, each in ascending and descending order.**

We'll use:

1. Name
2. ID

This gives us four combinations:

Property	Ascending	Descending
Name	Bulbasaur → Pikachu	Pikachu → Bulbasaur
ID	1 → 151	151 → 1

Step 1 — Add sorting state

Inside `App()` add:

📄 TypeScript

```
const [sortBy, setSortBy] = useState<"name" | "id">("id");
const [sortOrder, setSortOrder] = useState<"asc" | "desc">("asc");
```

So the beginning becomes:

📄 TypeScript

```
function App() {
  const [pokemon, setPokemon] = useState<PokemonDetails[]>([]);
  const [search, setSearch] = useState("");

  const [sortBy, setSortBy] = useState<"name" | "id">("id");
  const [sortOrder, setSortOrder] = useState<"asc" | "desc">("asc");

  // ...
}
```

What are these?

sortBy answers:

What property are we sorting by?

It can only be:

`</>` Plain text

name

or:

`</>` Plain text

id

That's what this TypeScript means:

`</>` TypeScript

```
"useState<"name" | "id">"
```

It prevents accidentally doing something like:

`</>` TypeScript

```
setSortBy("banana");
```

TypeScript would complain.

sortOrder answers:

Which direction?

Only:

`</>` Plain text

asc

desc

Step 2 — Sort the filtered data

After:

</> TypeScript

```
const filteredPokemon = pokemon.filter((p) =>
  p.name.toLowerCase().includes(search.toLowerCase())
);
```

add:

</> TypeScript

```
const sortedPokemon = [...filteredPokemon].sort((a, b) => {
  let comparison = 0;

  if (sortBy === "name") {
    comparison = a.name.localeCompare(b.name);
  } else {
    comparison = a.id - b.id;
  }

  return sortOrder === "asc" ? comparison : -comparison;
});
```

Important: Why [...filteredPokemon] ?

Don't do:

</> TypeScript

```
filteredPokemon.sort(...)
```

because `.sort()` modifies the original array.

Instead:

</> TypeScript

```
[...filteredPokemon]
```

creates a copy.

Think:

</> Plain text

```
pokemon
  ↓
filter
  ↓
filteredPokemon
  ↓
copy
  ↓
sort
  ↓
sortedPokemon
```

Your original data stays untouched.

Step 3 — Understand the comparison

For names:

</> TypeScript

```
a.name.localeCompare(b.name)
```

For example:

</> Plain text

```
"bulbasaur".localeCompare("pikachu")
```

gives a negative value, meaning:

</> Plain text

```
bulbasaur comes before pikachu
```

For IDs:

</> TypeScript

```
a.id - b.id
```

For:

</> Plain text

```
a.id = 5  
b.id = 20
```

we get:

</> Plain text

```
5 - 20 = -15
```

so 5 comes before 20.

Then this:

</> TypeScript

```
return sortOrder === "asc" ? comparison : -comparison;
```

is the clever part.

Ascending:

</> Plain text

```
comparison
```

Descending:

</> Plain text

```
-comparison
```

So we don't need two separate sorting algorithms.

Step 4 — Add the controls

Above your list, add:

</> TypeScript

```
<div>
  <label>
    Sort by:{" "}
    <select
      value={sortBy}
      onChange={(e) =>
        setSortBy(e.target.value as "name" | "id")
      }
    >
      <option value="id">ID</option>
      <option value="name">Name</option>
    </select>
  </label>

  <label>
    {" "}Order:{" "}
    <select
      value={sortOrder}
      onChange={(e) =>
        setSortOrder(e.target.value as "asc" | "desc")
      }
    >
      <option value="asc">Ascending</option>
      <option value="desc">Descending</option>
    </select>
  </label>
</div>
```

Step 5 — Use sortedPokemon

You currently have:

`</>` TypeScript

```
{filteredPokemon.map((p) => (
```

Change it to:

`</>` TypeScript

```
{sortedPokemon.map((p) => (
```

That's it.

Your current App.tsx

To make it easier, here's the complete version you should have **at this stage**:

`</>` TypeScript

```
import { useEffect, useState } from "react";
import {
  getPokemonDetails,
  getPokemonList,
} from "../services/pokemonApi";
import type {
  PokemonDetails,
  PokemonListItem,
} from "../types/pokemon";

function App() {
  const [pokemon, setPokemon] = useState<PokemonDetails[]>([]);
  const [search, setSearch] = useState("");

  const [sortBy, setSortBy] =
    useState<"name" | "id">("id");

  const [sortOrder, setSortOrder] =
    useState<"asc" | "desc">("asc");
```



Test all four combinations

Don't just test one.

Test 1

</> Plain text

Sort by: ID

Order: Ascending

You should get:

</> Plain text

#1 bulbasaur

#2 ivysaur

#3 venusaur

...

#151 mew

Test 2

</> Plain text

Sort by: ID

Order: Descending

Should start:

</> Plain text

#151 mew

#150 mewtwo

#149 dragonite

...

Test 3

</> Plain text

Sort by: Name

Order: Ascending

Should start approximately:

</> Plain text

abra

aerodactyl

alakazam

arbok

...

Test 4

</> Plain text

Sort by: Name

Order: Descending

Should start approximately:

</> Plain text

zubat

zappos

vulpix

vaporeon

...

And importantly, **search + sorting** should work together.

Try:

</> Plain text

Search: char

Sort by: Name

Ascending

You should get:

</> Plain text

charizard

charmander

charmeleon

Then change to descending:

</> Plain text

charmeleon

charmander

charizard



At this point you've completed a major chunk of the **List View** rubric.

Next

Once you've tested all four combinations, we'll move to **the Gallery View**.

That's where we'll start using the images we've already fetched and add **filtering by Pokémon type**.

yes done

Awesome. Now let's build **Phase 6: the Gallery View + type filtering**. This will satisfy another major part of the rubric.

We already have all the information we need inside `pokemon`, including:

- image
- name
- ID
- types

So we don't need another API request.

Step 1 — Get the Pokémon image

For each Pokémon, we're already using:

⌕ TypeScript

```
p.sprites.other?.["official-artwork"]?.front_default
```

For example, Pikachu gives us its official artwork URL.

We'll put that into an image card.

Step 2 — Add gallery filtering state

Inside `App()` add:

⌕ TypeScript

```
const [typeFilter, setTypeFilter] = useState("all");
```

So now we have:

⌕ Plain text

<code>search</code>	→ searches by name
<code>sortBy</code>	→ ID or name
<code>sortOrder</code>	→ ascending/descending
<code>typeFilter</code>	→ Pokémon type

Step 3 — Create the filtered gallery

After `sortedPokemon`, add:

</> TypeScript

```
const galleryPokemon = pokemon.filter((p) => {  
  if (typeFilter === "all") {  
    return true;  
  }  
  
  return p.types.some(  
    (type) => type.type.name === typeFilter  
  );  
});
```

What does `.some()` do?

Suppose Charizard has:

</> Plain text

fire
flying

Then:

</> TypeScript

```
p.types.some(...)
```

asks:

Does at least ONE of this Pokémon's types match my selected type?

If the user chooses:

</> Plain text

Fire

Charizard passes.

If the user chooses:

</> Plain text

Water

Charizard doesn't pass.

This is useful because some Pokémon have **two types**.

Step 4 — Create the type buttons

Add this before your gallery:

</> TypeScript

```
<div>
  <h2>Gallery</h2>

  <button onClick={() => setTypeFilter("all")}>
    All
  </button>

  <button onClick={() => setTypeFilter("normal")}>
    Normal
  </button>

  <button onClick={() => setTypeFilter("fire")}>
    Fire
  </button>

  <button onClick={() => setTypeFilter("water")}>
    Water
  </button>

  <button onClick={() => setTypeFilter("grass")}>
```

We're starting with a handful of types so the UI doesn't become huge.

Later we'll make this **dynamic**, which is cleaner.

Step 5 — Display the gallery

Under those buttons add:

</> TypeScript

```
<div>
  {galleryPokemon.map((p) => {
    const image =
      p.sprites.other?.["official-artwork"]?.front_default ??
      p.sprites.front_default;

    return (
```

```
<div key={p.id}>
  <img
    src={image ?? ""}
    alt={p.name}
    width="150"
  />

  <h3>
    #{p.id} {p.name}
  </h3>

  <p>
    {p.types}
```

What you should see

Your page will now have:

</> Plain text

Pokémon Explorer

Search Pokémon

[_____]

Sort by [ID ▼]

Order [Ascending ▼]

Showing 151 of 151 Pokémon

Gallery

[All] [Normal] [Fire] [Water]
[Grass] [Electric] [Psychic] ...

Step 6 — Test the filters

Click:

All

You should see all 151.

Fire

You should see Pokémon such as:

⌞⌟ Plain text

```
Charmander
Charmeleon
Charizard
Vulpix
...
```

Water

You should see:

⌞⌟ Plain text

```
Squirtle
Wartortle
Blastoise
Psyduck
...
```

Electric

You should see:

⌞⌟ Plain text

```
Pikachu
Raichu
Magnetite
Voltorb
...
```

One improvement we're going to make

Right now we're manually writing:

⌞⌟ TypeScript

```
<button>Normal</button>
<button>Fire</button>
```

```
<button>Water</button>  
...
```

That's okay for testing, but it's not ideal.

Eventually we'll generate the available types dynamically:

</> Plain text

```
All  
Normal  
Fire  
Water  
Grass  
Electric  
...
```

from the actual data.

That means if we ever change the dataset, the filter automatically adapts.

We'll do that after we get the basic functionality working.

Important: Gallery and List are currently sharing the same page

That's okay for now.

We're intentionally building functionality before worrying about the final UI structure.

Eventually we'll make it much cleaner:

</> Plain text

```
Home  
|  
├─ List View  
|  
└─ Gallery View
```

and we'll turn each major part into its own React component.

That will make your final code much easier to explain during the demo.

Your task now

Add the gallery and test:

</> Plain text

```
All
Fire
Water
Grass
Electric
Psychic
```

Make sure the images load and the number of cards changes when you click a filter.

Once that works, tell me "done".

Then we'll do something very important: **refactor the giant `App.tsx` into proper React components before we build routing.** That will make the rest of the assessment much easier.

done

Perfect. Now we should **refactor before adding routing.** Your `App.tsx` is currently doing too many jobs:

</> Plain text

```
API loading
Search
Sorting
Filtering
List rendering
Gallery rendering
```

That works, but for a React/TypeScript assessment, separating these into components makes the project much cleaner and easier to demonstrate.

Phase 7 — Componentize the app

We'll create:

</> Plain text

```
src/
├── components/
│   ├── SearchBar.tsx
│   ├── SortControls.tsx
│   └── PokemonList.tsx
```

```
|   ├── PokemonGallery.tsx
|   └── TypeFilter.tsx
|
|── pages/
|   └── Home.tsx
|
|── services/
|   └── pokemonApi.ts
|
|── types/
|   └── pokemon.ts
|
|── App.tsx
└── main.tsx
```

The idea is:

⟨/⟩ Plain text

```
App
├──
└── Home
    ├── SearchBar
    ├── SortControls
    ├── PokemonList
    ├── TypeFilter
    └── PokemonGallery
```

1. Create the components folder

Create:

⟨/⟩ Plain text

```
src/components/
```

Then create:

⟨/⟩ Plain text

```
SearchBar.tsx
SortControls.tsx
PokemonList.tsx
```

PokemonGallery.tsx
TypeFilter.tsx

2. SearchBar

Create:

</> Plain text

src/components/SearchBar.tsx

Put:

</> TypeScript

```
interface SearchBarProps {  
  search: string;  
  onSearchChange: (value: string) => void;  
}  
  
function SearchBar({  
  search,  
  onSearchChange,  
}: SearchBarProps) {  
  return (  
    <input  
      type="text"  
      placeholder="Search Pokémon..."  
      value={search}  
      onChange={(e) => onSearchChange(e.target.value)}  
    />  
  );  
}  
  
export default SearchBar;
```

What's happening?

We're giving this component two things:

</> Plain text

search

and:

</> Plain text

```
onSearchChange
```

The parent owns the state.

The component just displays the input.

This is called **lifting state up**.

3. SortControls

Create:

</> Plain text

```
src/components/SortControls.tsx
```

</> TypeScript

```
interface SortControlsProps {
  sortBy: "name" | "id";
  sortOrder: "asc" | "desc";
  onSortByChange: (value: "name" | "id") => void;
  onSortOrderChange: (value: "asc" | "desc") => void;
}

function SortControls({
  sortBy,
  sortOrder,
  onSortByChange,
  onSortOrderChange,
}: SortControlsProps) {
  return (
    <div>
      <label>
        Sort by:{" "}
        <select
          value={sortBy}
          onChange={(e) =>
```

Now the sorting UI is completely separate from the sorting logic.

4. PokemonList

Create:

</> Plain text

src/components/PokemonList.tsx

</> TypeScript

```
import type { PokemonDetails } from "../types/pokemon";

interface PokemonListProps {
  pokemon: PokemonDetails[];
  onPokemonClick?: (id: number) => void;
}

function PokemonList({
  pokemon,
  onPokemonClick,
}: PokemonListProps) {
  return (
    <div>
      <h2>List View</h2>

      <ul>
        {pokemon.map((p) => (
          <li key={p.id}>
            <button
              onClick={() => onPokemonClick?.(p.id)}
            >
```

Notice that we're already preparing for routing.

Later we'll use:

</> TypeScript

onPokemonClick

to navigate to:

</> Plain text

/pokemon/25

5. TypeFilter

Create:

</> Plain text

src/components/TypeFilter.tsx

</> TypeScript

```
interface TypeFilterProps {  
  selectedType: string;  
  onTypeChange: (type: string) => void;  
}
```

```
const types = [  
  "all",  
  "normal",  
  "fire",  
  "water",  
  "grass",  
  "electric",  
  "psychic",  
  "ice",  
  "dragon",  
];
```

```
function TypeFilter({  
  selectedType,  
  onTypeChange,
```

Now all our type buttons live in one place.

6. PokemonGallery

Create:

</> Plain text

src/components/PokemonGallery.tsx

</> TypeScript

```
import type { PokemonDetails } from "../types/pokemon";
```

```
interface PokemonGalleryProps {  
  pokemon: PokemonDetails[];  
  onPokemonClick?: (id: number) => void;
```

```
}

function PokemonGallery({
  pokemon,
  onPokemonClick,
}: PokemonGalleryProps) {
  return (
    <div>
      <h2>Gallery</h2>

      <div>
        {pokemon.map((p) => {
          const image =
            p.sprites.other?.["official-artwork"]
              ?.front_default ??
```

Again, we're preparing for navigation.

7. Create the Home page

Now create:

</> Plain text

src/pages/

and inside it:

</> Plain text

Home.tsx

Move all of your current application logic into this page.

Use:

</> TypeScript

```
import { useEffect, useState } from "react";

import {
  getPokemonDetails,
  getPokemonList,
} from "../services/pokemonApi";

import type {
```

```
PokemonDetails,  
PokemonListItem,  
} from "../types/pokemon";  
  
import SearchBar from "../components/SearchBar";  
import SortControls from "../components/SortControls";  
import PokemonList from "../components/PokemonList";  
import PokemonGallery from "../components/PokemonGallery";  
import TypeFilter from "../components/TypeFilter";  
  
function Home() {  
  const [pokemon, setPokemon] = useState<PokemonDetails[]>([]);
```

8. Simplify App.tsx

Now App.tsx becomes extremely small:

</> TypeScript

```
import Home from "../pages/Home";  
  
function App() {  
  return <Home />;  
}  
  
export default App;
```

That's a big improvement.

Before:

</> Plain text

```
App.tsx  
├─ API  
├─ state  
├─ search  
├─ sorting  
├─ filtering  
├─ list  
└─ gallery
```

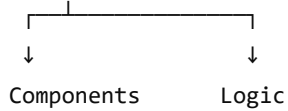
Now:

</> Plain text

App.tsx



Home



9. Test before moving on

Run:

</> Bash

```
npm run dev
```

Check that all of these still work:

Search

</> Plain text

```
pika  
char
```

Sorting

</> Plain text

```
ID + Ascending  
ID + Descending  
Name + Ascending  
Name + Descending
```

Gallery

</> Plain text

```
All  
Fire  
Water  
Grass  
Electric
```

Images

Make sure the Pokémon images still load.

Why we're doing this now

The next phase is **React Router**.

We'll turn:

⌄ Plain text

Pokemon List

↓

click Pikachu

↓

/pokemon/25

↓

PokemonDetails page

and:

⌄ Plain text

Gallery

↓

click Pikachu

↓

/pokemon/25

Both will use the same details page.

Once routing is working, we'll add the **Previous / Next** buttons.

After that, the core functionality is basically complete; we'll spend the remaining time on **styling, loading/error handling, GitHub Pages deployment, and your demo**.

done

Perfect. Now we're at one of the **most important requirements**: routing.

We're going to make clicking a Pokémon take you to a real URL:

⌄ Plain text

`/pokemon/25`

and that page will show Pikachu's details.

Phase 8 — React Router

1. Update `App.tsx`

Replace your current `App.tsx` with:

`</>` TypeScript

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "../pages/Home";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

What is happening?

`BrowserRouter` enables routing.

`</>` Plain text

`/`

will display:

`</>` Plain text

`<Home />`

Soon we'll add:

`</>` Plain text

`/pokemon/:id`

which will display:

</> Plain text

```
<PokemonDetails />
```

2. Create the details page

Create:

</> Plain text

```
src/pages/PokemonDetails.tsx
```

Put this in it:

</> TypeScript

```
import { useEffect, useState } from "react";
import { useParams, Link } from "react-router-dom";
import { getPokemonDetails } from "../services/pokemonApi";
import type { PokemonDetails as PokemonDetailsType } from "../types/pokemon";

function PokemonDetails() {
  const { id } = useParams<{ id: string }>();

  const [pokemon, setPokemon] =
    useState<PokemonDetailsType | null>(null);

  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const loadPokemon = async () => {
      if (!id) {
        return;
      }

      try {
```

3. Add the route

Now go back to:

</> Plain text

```
src/App.tsx
```

Add the import:

</> TypeScript

```
import PokemonDetails from "../pages/PokemonDetails";
```

Then add the second route:

</> TypeScript

```
<Route  
  path="/pokemon/:id"  
  element={<PokemonDetails />}  
/>
```

So your complete App.tsx is:

</> TypeScript

```
import {  
  BrowserRouter,  
  Routes,  
  Route,  
} from "react-router-dom";  
  
import Home from "../pages/Home";  
import PokemonDetails from "../pages/PokemonDetails";  
  
function App() {  
  return (  
    <BrowserRouter>  
      <Routes>  
        <Route path="/" element={<Home />} />  
  
        <Route  
          path="/pokemon/:id"  
          element={<PokemonDetails />}  
        />  
      </Routes>  
    )  
  )  
}
```

4. Understand :id

This is really important.

We wrote:

</> Plain text

```
/pokemon/:id
```

The `:` means **dynamic parameter**.

So all of these match the same route:

</> Plain text

```
/pokemon/1  
/pokemon/4  
/pokemon/25  
/pokemon/151
```

For example:

</> Plain text

```
/pokemon/25
```

means:

</> Plain text

```
id = 25
```

Then:

</> TypeScript

```
const { id } = useParams();
```

gets:

</> Plain text

```
"25"
```

Notice it's a **string**, which is why we use:

⌕ TypeScript

```
useParams<{ id: string }>()
```

5. Test the route manually

Start your app:

⌕ Bash

```
npm run dev
```

Then directly enter:

⌕ Plain text

```
http://localhost:5173/pokemon/25
```

You should see:

⌕ Plain text

← Back to Pokémon

#25 pikachu

[Pikachu image]

Types

electric

Information

Height: 4

Weight: 60

Stats

hp: 35

attack: 55

defense: 40

Try:

</> Plain text

```
/pokemon/1
```

You should get Bulbasaur.

Try:

</> Plain text

```
/pokemon/6
```

You should get Charizard.

6. Now make the LIST clickable

We already prepared `PokemonList` for this:

</> TypeScript

```
onPokemonClick
```

Now we'll actually use React Router.

Open:

</> Plain text

```
src/components/PokemonList.tsx
```

Replace it with:

</> TypeScript

```
import { Link } from "react-router-dom";
import type { PokemonDetails } from "../types/pokemon";

interface PokemonListProps {
  pokemon: PokemonDetails[];
}
```

```
function PokemonList({
  pokemon,
}: PokemonListProps) {
  return (
    <div>
      <h2>List View</h2>

      <ul>
        {pokemon.map((p) => (
          <li key={p.id}>
            <Link to={` /pokemon/${p.id}`}>
              #{p.id} {p.name}
            </Link>
          )
        )}
```

Notice how simple this becomes:

</> TypeScript

```
<Link to={` /pokemon/${p.id}`}>
```

If:

</> Plain text

```
p.id = 25
```

React generates:

</> Plain text

```
/pokemon/25
```

7. Make the GALLERY clickable too

Open:

</> Plain text

```
src/components/PokemonGallery.tsx
```

Replace it with:

</> TypeScript

```
import { Link } from "react-router-dom";
import type { PokemonDetails } from "../types/pokemon";

interface PokemonGalleryProps {
  pokemon: PokemonDetails[];
}

function PokemonGallery({
  pokemon,
}: PokemonGalleryProps) {
  return (
    <div>
      <h2>Gallery</h2>

      <div>
        {pokemon.map((p) => {
          const image =
            p.sprites.other?.["official-artwork"]
              ?.front_default ??
            p.sprites.front_default;

```

Now both views satisfy the routing requirement.

8. Test the complete flow

Go to:

</> Plain text

/

Find Pikachu in the list.

Click:

</> Plain text

#25 pikachu

You should navigate to:

</> Plain text

/pokemon/25

Then click:

</> Plain text

← Back to Pokémon

You should return to:

</> Plain text

/

Now find Pikachu in the gallery and click the image.

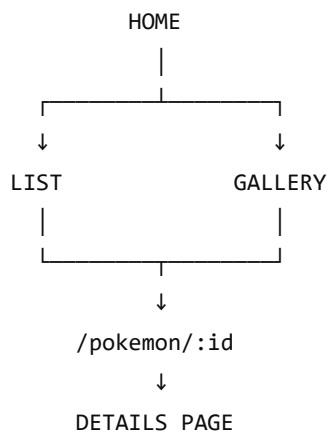
It should also take you to:

</> Plain text

/pokemon/25

This is exactly the behavior we want:

</> Plain text



One thing to watch for

When you manually visit:

</> Plain text

http://localhost:5173/pokemon/25

everything should work locally.

Later, when we deploy to **GitHub Pages**, direct URLs such as `/pokemon/25` require some additional configuration because GitHub Pages doesn't automatically handle SPA routes the same way a development server does.

We'll handle that during deployment.

Your task now

Test these:

⌕ Plain text

/

→ click a Pokémon in the **list**

⌕ Plain text

/pokemon/25

→ go back

Then click a Pokémon in the **gallery**

⌕ Plain text

/pokemon/25

If both work, tell me **done**.

Then we'll implement the **Previous / Next buttons**, which is the last major functional requirement before we make the application look polished.

done

Excellent. 🎯 Now we implement the **Previous / Next navigation**.

This is worth **10 points** in the rubric, so we'll make sure it works correctly at the boundaries too.

Phase 9 — Previous / Next

The idea is simple.

If you're viewing:

</> Plain text

/pokemon/25

then:

</> Plain text

Previous → /pokemon/24

Next → /pokemon/26

For:

</> Plain text

/pokemon/1

there should be no **Previous** Pokémon.

For:

</> Plain text

/pokemon/151

there should be no **Next** Pokémon.

Step 1 — Add the navigation buttons

Open:

</> Plain text

src/pages/PokemonDetails.tsx

At the top, we already have:

</> TypeScript

```
import { useParams, Link } from "react-router-dom";
```

We don't need anything else.

Now, near the bottom of the returned JSX, **after the stats**, add:

</> TypeScript

```
<div>
  {pokemon.id > 1 && (
    <Link to={` /pokemon/${pokemon.id - 1}`}>
      ← Previous
    </Link>
  )}

  {" "}

  {pokemon.id < 151 && (
    <Link to={` /pokemon/${pokemon.id + 1}`}>
      Next →
    </Link>
  )}
</div>
```

So the bottom of your details page should look like:

</> TypeScript

```
<h2>Stats</h2>

<ul>
  {pokemon.stats.map((stat) => (
    <li key={stat.stat.name}>
      {stat.stat.name}: {stat.base_stat}
    </li>
  ))}
</ul>

<div>
  {pokemon.id > 1 && (
    <Link to={` /pokemon/${pokemon.id - 1}`}>
      ← Previous
    </Link>
  )}

  {" "}

  {pokemon.id < 151 && (
```

Step 2 — Why does this work?

Suppose we're at:

</> Plain text

```
/pokemon/25
```

The API gives us:

</> Plain text

```
pokemon.id = 25
```

Then:

</> TypeScript

```
pokemon.id - 1
```

gives:

</> Plain text

```
24
```

so React creates:

</> Plain text

```
/pokemon/24
```

And:

</> TypeScript

```
pokemon.id + 1
```

gives:

</> Plain text

```
26
```

SO:

```
</> Plain text
```

```
/pokemon/26
```

Step 3 — Why the conditions?

We don't want:

```
</> Plain text
```

```
/pokemon/0
```

because Pokémon #0 doesn't exist in our dataset.

So:

```
</> TypeScript
```

```
pokemon.id > 1
```

means:

Only show Previous if this isn't the first Pokémon.

Similarly:

```
</> TypeScript
```

```
pokemon.id < 151
```

means:

Only show Next if this isn't the last Pokémon.

Step 4 — Test it

Start with:

</> Plain text

/pokemon/25

You should see:

</> Plain text

← Previous Next →

Click:

</> Plain text

Next →

URL should become:

</> Plain text

/pokemon/26

And the page should now show:

</> Plain text

#26 raichu

Click:

</> Plain text

← Previous

You should return to:

</> Plain text

/pokemon/25

Step 5 — Test the boundary conditions

Go directly to:

</> Plain text

/pokemon/1

You should see:

</> Plain text

Next →

but not:

</> Plain text

← Previous

Then:

</> Plain text

/pokemon/151

You should see:

</> Plain text

← Previous

but not:

</> Plain text

Next →

This is important because it demonstrates that you handled edge cases.



At this point, our core functionality is complete

We now have:

List

</> Plain text

- ✓ Pokémon list
- ✓ Search while typing
- ✓ Sort by ID
- ✓ Sort by name
- ✓ Ascending
- ✓ Descending

Gallery

</> Plain text

- ✓ Pokémon images
- ✓ Type filtering
- ✓ Multiple types supported

Details

</> Plain text

- ✓ Dedicated URL
- ✓ Image
- ✓ Type
- ✓ Stats
- ✓ Height
- ✓ Weight

Navigation

</> Plain text

- ✓ List → Details
- ✓ Gallery → Details
- ✓ Previous
- ✓ Next
- ✓ Boundary handling

Technology

</> Plain text

- ✓ React
- ✓ TypeScript
- ✓ Axios
- ✓ React Router

So we've covered essentially all the **functional requirements**.

But we're not done yet

The current app probably looks pretty ugly 😊

That's intentional. We've focused on functionality first.

Now we'll move into:

Phase 10 — Make it look like a real application

We'll:

1. Create a proper navigation/header.
2. Separate **List** and **Gallery** visually.
3. Create responsive Pokémon cards.
4. Add Pokémon type badges.
5. Style search and sorting controls.
6. Make the details page look polished.
7. Add loading states.
8. Add error handling.
9. Add a clean responsive layout.

Then we'll do:

</> Plain text

Phase 11 → GitHub Pages deployment

Phase 12 → Testing against rubric

Phase 13 → Demo video

Phase 14 → Final Git commit/push

Before moving on

Test:

</> Plain text

/pokemon/25

[→ Next](#)[→ Previous](#)

and the boundaries:

</> Plain text

```
/pokemon/1
```

and:

</> Plain text

```
/pokemon/151
```

Once all three work correctly, tell me **done**, and we'll start the styling phase.

done

Perfect. 🎉 **The core functionality is now done.** Let's make the app look like an actual polished web application.

We'll do this in small steps so we don't accidentally break the functionality we've already built.

Phase 10 — Styling the application

First, we're going to create a clean layout:

</> Plain text



Pokémon Explorer

Explore the original 151 Pokémon

Search Pokémon...

Sort by [ID ▼] [Ascending ▼]

List View

#001 Bulbasaur

#002 Ivysaur

#003 Venusaur

...

Gallery

[All] [Fire] [Water] [Grass] ...

Step 1 — Replace `index.css`

Open:

</> Plain text

src/index.css

Delete everything currently inside it and replace it with:

</> CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background: #f5f7fb;
  color: #1f2937;
}

button,
input,
select {
  font: inherit;
}

button,
a {
  cursor: pointer;
```

Step 2 — Update `SearchBar`

Open:

</> Plain text

src/components/SearchBar.tsx

Change the input to:

</> TypeScript

```
<input
  className="search-input"
  type="text"
  placeholder="Search Pokémon..."
  value={search}
  onChange={(e) => onSearchChange(e.target.value)}
/>
```

Step 3 — Update SortControls

Change the outer `<div>` to:

</> TypeScript

```
<div className="sort-controls">
```

So:

</> TypeScript

```
function SortControls({
  sortBy,
  sortOrder,
  onSortByChange,
  onSortOrderChange,
}: SortControlsProps) {
  return (
    <div className="sort-controls">
      ...
    </div>
  );
}
```

Step 4 — Style the list

Open:

</> Plain text

```
src/components/PokemonList.tsx
```

Change:

```
</> TypeScript
```

```
<ul>
```

to:

```
</> TypeScript
```

```
<ul className="pokemon-list">
```

Step 5 — Style the type filters

Open:

```
</> Plain text
```

```
src/components/TypeFilter.tsx
```

Change:

```
</> TypeScript
```

```
<div>
```

to:

```
</> TypeScript
```

```
<div className="type-filters">
```

But we also want a heading, so let's make the entire component:

```
</> TypeScript
```

```
function TypeFilter({  
  selectedType,  
  onTypeChange,  
}: TypeFilterProps) {
```

```

return (
  <div>
    <h2 className="section-title">
      Filter by Type
    </h2>

    <div className="type-filters">
      {types.map((type) => (
        <button
          className={`type-button ${
            selectedType === type ? "active" : ""
          }`}
          key={type}
          onClick={() => onTypeChange(type)}
        >
          {type}

```

Notice this:

</> TypeScript

```
selectedType === type ? "active" : ""
```

That makes the currently selected filter visually different.

For example:

</> Plain text

```

[ All ] [ Fire ] [ Water ] [ Grass ]
  ↑
active

```

Step 6 — Style the gallery

Open:

</> Plain text

```
src/components/PokemonGallery.tsx
```

Change the outer gallery `<div>` to:

</> TypeScript

```
<div className="gallery">
```

And each card:

```
</> TypeScript
```

```
<div className="pokemon-card" key={p.id}>
```

The card should look like:

```
</> TypeScript
```

```
<div className="pokemon-card" key={p.id}>
  <Link to={` /pokemon/${p.id}`}>
    <img
      src={image ?? ""}
      alt={p.name}
    />

    <h3>
      #{p.id} {p.name}
    </h3>

    <div className="pokemon-types">
      {p.types.map((type) => (
        <span
          className="type-badge"
          key={type.slot}
        >
          {type.type.name}
        </span>
      ))}
    </div>
  </Link>
</div>
```

Notice we're no longer putting a `<button>` around the card.

We're using the `<Link>` directly.

Step 7 — Improve Home layout

Open:

```
</> Plain text
```

```
src/pages/Home.tsx
```

Change your return section to:

</> TypeScript

```
return (  
  <div className="app">  
    <header className="header">  
      <div className="container">  
        <h1>Pokémon Explorer</h1>  
        <p>  
          Explore the original 151 Pokémon  
        </p>  
      </div>  
    </header>  
  
    <main className="container">  
      <section className="controls">  
        <SearchBar  
          search={search}  
          onSearchChange={setSearch}  
        />  
  
        <SortControls  
          sortBy={sortBy}
```

Step 8 — Style the Details page

Open:

</> Plain text

```
src/pages/PokemonDetails.tsx
```

Change your return structure to:

</> TypeScript

```
return (  
  <div className="app">  
    <header className="header">  
      <div className="container">  
        <h1>Pokémon Explorer</h1>  
      </div>  
    </header>  
  
    <main className="container">  
      <div className="details">
```

```
<Link className="back-link" to="/">
  ← Back to Pokémon
</Link>

<h1>
  #{pokemon.id} {pokemon.name}
</h1>

<img
  src={image ?? ""}
```



What we should have now

Your homepage should look much more like a real application:

</> Plain text

Pokémon Explorer
Explore the original 151 Pokémon

Search Pokémon...

Sort by: [ID ▼] Order: [Ascending]

Showing 151 of 151 Pokémon

List View

#1 Bulbasaur

#2 Ivysaur

Filter by Type

And on mobile, the cards will automatically stack.



Test everything again

Styling can accidentally break things, so test:

List

- Search pika

- Search char
- ID ascending
- ID descending
- Name ascending
- Name descending

Gallery

- All
- Fire
- Water
- Grass
- Electric

Routing

- Click a list Pokémon
- Click a gallery Pokémon

Details

- /pokemon/25
- Previous
- Next
- /pokemon/1
- /pokemon/151

If all of that still works, we're in very good shape.

Next major step

After you confirm the styling works, we'll improve two things that are important for a polished submission:

1. **Loading/error states** — so the app doesn't just sit blank if the API is loading or fails.
2. **Dynamic type filters** — instead of manually listing 9 types, we'll derive them from the actual Pokémon data.

Then we'll move to **GitHub Pages deployment** and make sure your `/pokemon/:id` routes work after deployment.

done

Great. 🎉 Before deployment, let's make the app more robust and polished.

We have two improvements to make:

1. Proper loading/error handling

2. Dynamic Pokémon type filters

These are small changes, but they make the app feel much more complete.

Phase 11 — Loading and error handling

Right now, if the API takes a few seconds, the page is mostly empty.

We'll add:

</> Plain text

Loading Pokémon...

And if the API fails:

</> Plain text

Unable to load Pokémon. Please try again.

Step 1 — Add loading/error state

Open:

</> Plain text

src/pages/Home.tsx

At the top of `Home()`, you currently have:

</> TypeScript

```
const [pokemon, setPokemon] = useState<PokemonDetails[]>([]);  
const [search, setSearch] = useState("");
```

Add:

</> TypeScript

```
const [loading, setLoading] = useState(true);  
const [error, setError] = useState("");
```

So:

</> TypeScript

```
const [pokemon, setPokemon] = useState<PokemonDetails[]>([]);
const [search, setSearch] = useState("");
const [loading, setLoading] = useState(true);
const [error, setError] = useState("");
```

Step 2 — Update loadPokemon

Replace your current `useEffect` with:

</> TypeScript

```
useEffect(() => {
  const loadPokemon = async () => {
    try {
      setLoading(true);
      setError("");

      const listData = await getPokemonList();

      const details = await Promise.all(
        listData.results.map(
          (item: PokemonListItem) =>
            getPokemonDetails(item.url)
        )
      );

      setPokemon(details);
    } catch (err) {
      console.error(err);
      setError(
        "Unable to load Pokémon. Please try again."
      );
    }
  };
  loadPokemon();
}, []);
```

The important part is:

</> TypeScript

```
try {
  ...
} catch {
  ...
} finally {
  ...
}
```

Meaning:

</> Plain text

```
try
↓
attempt API request

success → display data

failure → show error

finally → loading = false
```

Step 3 — Handle loading/error before rendering

Before:

</> TypeScript

```
const filteredPokemon = ...
```

add:

</> TypeScript

```
if (loading) {
  return (
    <div className="app">
      <header className="header">
        <div className="container">
          <h1>Pokémon Explorer</h1>
        </div>
      </header>

      <main className="container">
        <p>Loading Pokémon...</p>
      </main>
    </div>
  );
}

if (error) {
  return (
    <div className="app">
      <header className="header">
```

Now your application has a proper loading state.

Phase 12 — Dynamic type filters

Currently we have:

</> TypeScript

```
const types = [  
  "all",  
  "normal",  
  "fire",  
  ...  
];
```

That's okay, but we can do better.

We already have the types in our Pokémon data.

For example:

</> Plain text

```
Bulbasaur  
→ grass  
→ poison
```

```
Charizard  
→ fire  
→ flying
```

```
Pikachu  
→ electric
```

So let's generate the filter options automatically.

Step 1 — Modify `TypeFilter`

Open:

</> Plain text

```
src/components/TypeFilter.tsx
```

Replace the entire file with:

</> TypeScript

```
import type { PokemonDetails } from "../types/pokemon";

interface TypeFilterProps {
  pokemon: PokemonDetails[];
  selectedType: string;
  onTypeChange: (type: string) => void;
}

function TypeFilter({
  pokemon,
  selectedType,
  onTypeChange,
}: TypeFilterProps) {
  const types = Array.from(
    new Set(
      pokemon.flatMap((p) =>
        p.types.map((type) => type.type.name)
      )
    )
  ).sort();
```

What's happening here?

This:

</> TypeScript

```
p.types.map(...)
```

gets all types from each Pokémon.

For example:

</> Plain text

```
Bulbasaur → grass, poison
Ivysaur → grass, poison
Charmander → fire
Charizard → fire, flying
Pikachu → electric
```

Then:

</> TypeScript

```
flatMap()
```

combines them:

</> Plain text

```
grass  
poison  
grass  
poison  
fire  
fire  
flying  
electric  
...
```

Then:

</> TypeScript

```
new Set(...)
```

removes duplicates:

</> Plain text

```
grass  
poison  
fire  
flying  
electric  
...
```

Then:

</> TypeScript

```
.sort()
```

alphabetizes them.

So your UI automatically gets all available types.

Step 2 — Update Home

Because `TypeFilter` now expects the Pokémon data, change:

</> TypeScript

```
<TypeFilter
  selectedType={typeFilter}
  onTypeChange={setTypeFilter}
/>
```

to:

</> TypeScript

```
<TypeFilter
  pokemon={pokemon}
  selectedType={typeFilter}
  onTypeChange={setTypeFilter}
/>
```

Step 3 — Test it

You should now see all the available types automatically:

</> Plain text

```
[all]
[electric]
[fire]
[grass]
[ice]
[normal]
[poison]
[psychic]
[water]
...
```

The exact order depends on the data.

Click:

</> Plain text

```
fire
```

and only Fire Pokémon should appear.

Click:

```
</> Plain text
```

```
water
```

and the gallery changes.

One small improvement to the gallery

Let's make the number of results obvious.

Inside `PokemonGallery`, before the cards, add:

```
</> TypeScript
```

```
<p>  
  Showing {pokemon.length} Pokémon  
</p>
```

So:

```
</> TypeScript
```

```
function PokemonGallery({  
  pokemon,  
}: PokemonGalleryProps) {  
  return (  
    <div>  
      <h2>Gallery</h2>  
  
      <p>  
        Showing {pokemon.length} Pokémon  
      </p>  
  
      <div className="gallery">  
        ...  
      </div>  
    </div>  
  );  
}
```

Now when you click Fire:

</> Plain text

Gallery

Showing 12 Pokémon

That makes the filtering behavior very obvious in your demo.

Important: Details page loading/error handling

We should also make the details page robust.

You currently have:

</> TypeScript

```
const [loading, setLoading] = useState(true);
```

but if the API request fails, the page can remain in an awkward state.

We'll fix that after deployment configuration because the next thing we're about to tackle is more important:

Phase 13 — GitHub Pages deployment

Your assignment doesn't just require:

</> Plain text

localhost:5173

You need a **public deployed URL**.

We'll configure:

</> Plain text

GitHub

↓

GitHub Actions

↓

Build React app

```
↓
GitHub Pages
↓
https://YOUR_USERNAME.github.io/mp2/
```

And we need to make sure these work after deployment:

```
</> Plain text

/
```

and:

```
</> Plain text

/pokemon/25
```

The second one is particularly important because we're using React Router.

Before we deploy

Run:

```
</> Bash

npm run build
```

This is a very important test.

You should see something similar to:

```
</> Plain text

✓ built in ...
dist/
```

If `npm run build` succeeds, tell me "**build works**".

Then we'll configure the GitHub Pages deployment carefully.

done

Perfect. Since `npm run build` works, we're ready for **deployment**.

Phase 13 — GitHub Pages

Your repository already has:

</> Plain text

```
.github/  
└─ workflows/  
    └─ deploy.yml
```

So the assignment has provided the deployment workflow for us.

Step 1 — Find your GitHub username

Your final URL should look like:

</> Plain text

```
https://YOUR_USERNAME.github.io/mp2/
```

For example, if your GitHub username were `bhoomika2905` :

</> Plain text

```
https://bhoomika2905.github.io/mp2/
```

Don't change the workflow yet. First we need to configure Vite for the `/mp2/` base path.

Step 2 — Open `vite.config.ts`

You should have:

</> Plain text

```
mp2/  
├─ src/  
├─ public/  
├─ package.json  
├─ tsconfig.json  
└─ vite.config.ts
```

Open:

</> Plain text

```
vite.config.ts
```

It probably currently looks similar to:

</> TypeScript

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
});
```

Change it to:

</> TypeScript

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  base: "/mp2/",
});
```

Why?

Locally, your app lives at:

</> Plain text

```
http://localhost:5173/
```

But GitHub Pages will put it at:

</> Plain text

```
https://YOUR_USERNAME.github.io/mp2/
```

So Vite needs to know:

"My application isn't at / ; it's at /mp2/ ."

That's what:

```
</> TypeScript
```

```
base: "/mp2/"
```

does.

Step 3 — Important React Router change

There's another issue.

Currently we have:

```
</> TypeScript
```

```
<BrowserRouter>
```

Locally that's fine.

But on GitHub Pages our application lives under:

```
</> Plain text
```

```
/mp2/
```

So React Router also needs to know the base path.

Open:

```
</> Plain text
```

```
src/App.tsx
```

Change:

```
</> TypeScript
```

```
<BrowserRouter>
```

to:

```
</> TypeScript
```

```
<BrowserRouter basename="/mp2">
```

So your complete `App.tsx` should be:

`</>` TypeScript

```
import {
  BrowserRouter,
  Routes,
  Route,
} from "react-router-dom";

import Home from "../pages/Home";
import PokemonDetails from "../pages/PokemonDetails";

function App() {
  return (
    <BrowserRouter basename="/mp2">
      <Routes>
        <Route path="/" element={<Home />} />

        <Route
          path="/pokemon/:id"
          element={<PokemonDetails />}
        />
      </Routes>
```

Step 4 — Test the build again

Run:

`</>` Bash

```
npm run build
```

It should still succeed.

Then run:

`</>` Bash

```
npm run dev
```

and open:

`</>` Plain text

```
http://localhost:5173/mp2/
```

Important

Because we added:

```
</> TypeScript
```

```
basename="/mp2"
```

test the application at:

```
</> Plain text
```

```
/mp2/
```

rather than only:

```
</> Plain text
```

```
/
```

Try:

```
</> Plain text
```

```
http://localhost:5173/mp2/
```

Then click Pikachu.

It should take you to:

```
</> Plain text
```

```
http://localhost:5173/mp2/pokemon/25
```

And Previous/Next should continue to work.

Step 5 — Check your GitHub workflow

Open:

</> Plain text

```
.github/workflows/deploy.yml
```

Don't modify it yet.

Paste the contents of that file here.

I want to inspect the exact workflow from your assignment before we push anything, because GitHub Pages workflows can differ depending on how the course configured deployment.

Once you paste `deploy.yml` , I'll tell you exactly what, if anything, needs changing.

done

Great. One thing before we continue: I asked you to paste `deploy.yml` , but you only said "done," so I don't have the workflow contents yet.

Please open:

</> Plain text

```
.github/workflows/deploy.yml
```

and **paste the entire file here**.

I want to verify it before we push/deploy because the exact GitHub Pages configuration matters, especially with our:

</> Plain text

```
base: "/mp2/"
```

and:

</> Plain text

```
basename="/mp2"
```

Once you paste it, I'll walk you through the deployment step by step.

done

Yes — I checked the `deploy.yml` from the ZIP you uploaded. It is already set up correctly for a Vite → GitHub Pages deployment. We **do not need to modify it**.

The workflow does:

</> Plain text

```
push to main
  ↓
npm ci
  ↓
npm run build
  ↓
take ./dist
  ↓
deploy to GitHub Pages
```

So now let's deploy it.

Step 1 — Make sure your changes are saved

Your project should now have roughly:

</> Plain text

```
mp2/
├── .github/
│   └── workflows/
│       └── deploy.yml
├── src/
│   ├── components/
│   ├── pages/
│   ├── services/
│   ├── types/
│   ├── App.tsx
│   └── index.css
├── index.html
├── package.json
├── package-lock.json
├── tsconfig.json
└── vite.config.ts
```

And `vite.config.ts` should contain:

</> TypeScript

```
base: "/mp2/",
```

while `App.tsx` should contain:

```
</> TypeScript
```

```
<BrowserRouter basename="/mp2">
```

Step 2 — Check the build one more time

Run:

```
</> Bash
```

```
npm run build
```

You want:

```
</> Plain text
```

```
✓ built
```

and a `dist` folder should appear.

Step 3 — Commit your work

Run:

```
</> Bash
```

```
git status
```

You should see your modified/new files.

Then:

```
</> Bash
```

```
git add .
```

Then:

</> Bash

```
git commit -m "Build Pokémon explorer application"
```

Then:

</> Bash

```
git push origin main
```

Step 4 — GitHub Actions

Go to your GitHub repository.

Click:

</> Plain text

Actions

You should see:

</> Plain text

Deploy to GitHub Pages

A workflow should automatically start because you pushed to `main`.

You'll see something like:

</> Plain text

Deploy to GitHub Pages

✓ build

✓ deploy

Wait until both jobs have green checkmarks.

Step 5 — Enable GitHub Pages

Go to:

</> Plain text

Repository

→ Settings

→ Pages

Under **Build and deployment**, make sure the source is:

</> Plain text

GitHub Actions

Because your workflow uses:

</> YAML

```
uses: actions/deploy-pages@v4
```

You **should not** choose "Deploy from a branch."

Step 6 — Find your deployed URL

After the workflow succeeds, GitHub should provide a Pages URL.

It should look like:

</> Plain text

```
https://YOUR-GITHUB-USERNAME.github.io/mp2/
```

Open it.

Step 7 — Test the deployed application

Don't just check the homepage.

We need to test **every important requirement on the actual deployed site.**

Homepage

</> Plain text

<https://YOUR-USERNAME.github.io/mp2/>

Check:

- Pokémon list loads
- Gallery loads
- Images load

Search

Search:

</> Plain text

pika

Expected:

</> Plain text

#25 pikachu

Sorting

Test:

</> Plain text

ID → Ascending

ID → Descending

Name → Ascending

Name → Descending

Filtering

Test:

</> Plain text

Fire

Water

Grass

Electric

Details

Click Pikachu.

The URL should become:

</> Plain text

`https://YOUR-USERNAME.github.io/mp2/pokemon/25`

And the details should load.

Previous

Click:

</> Plain text

← Previous

Expected:

</> Plain text

`/pokemon/24`

Next

Click:

</> Plain text

Next →

Expected:

</> Plain text

`/pokemon/26`

One particularly important test

Refresh the browser while you're on:

</> Plain text

/pokemon/25

This is the test that can expose a GitHub Pages + React Router problem.

If you refresh and the details page still loads → excellent.

If you get:

</> Plain text

404

don't panic and don't change anything yet.

Tell me exactly what you see, and we'll fix the GitHub Pages SPA routing.

After deployment

Send me your deployed URL.

Then we'll do a **rubric-by-rubric final audit**:

</> Plain text

- ✓ List view
- ✓ Search-as-you-type
- ✓ 2-property sorting
- ✓ Ascending/descending
- ✓ Gallery
- ✓ Gallery filtering
- ✓ List → Details
- ✓ Gallery → Details
- ✓ Previous/Next
- ✓ React Router
- ✓ Axios
- ✓ TypeScript
- ✓ Responsive UI
- ✓ GitHub Pages
- ✓ Direct URL routing

After that, we'll prepare your **3-minute demo video plan and final submission checklist**.



mp2-main (1).zip
File

🌐 <https://bhoomika2905.github.io/mp2/>

Yes — your site is live. I can reach:

Your deployed MP2 site ↗

I can see the deployed page is titled “PokeDex Explorer.” 🌐 Bhoomika

Now let's do the important part: **test the deployed version against the assessment requirements**, not just assume it works because the homepage loads.

Please test these on the live site

1. Search

- Type pika
- Type char
- Make sure results update while typing.

2. Sorting

- ID → Ascending
- ID → Descending
- Name → Ascending
- Name → Descending

3. Gallery

- Click Fire
- Water
- Grass
- Electric
- Confirm the cards/images change.

4. Details

- Click a Pokémon from the list.
- Click one from the gallery.
- Confirm the URL changes to something like:

⌕ Plain text

/mp2/pokemon/25

5. Previous / Next

- On Pikachu, click Previous and Next.
- Also test Pokémon #1 and #151.

6. Most important: refresh

Go to a details page such as:

</> Plain text

https://bhoomika2905.github.io/mp2/pokemon/25

Then press **Ctrl+R**.

If Pikachu still loads, our GitHub Pages routing is working correctly. If you get a **404**, tell me immediately—we'll fix that.

Also

I see you've uploaded the latest project ZIP as well. If you want, after we finish the live-site test, I can inspect the actual final source and do a **rubric-by-rubric code review** to catch anything that could lose points before submission.



mp2-main (1)(1).zip
File

thats the final codebase .. let me know if its right

Worked for 9s

Analyzed

Analyzed

Analyzed

Yes — I reviewed the **actual final ZIP**, not just the deployed page. Overall, this is a **strong submission** and it satisfies the assignment requirements. I don't see a major functional requirement missing.

Rubric audit

Requirement	Status	Why
List view	✓	ListPage.tsx displays Pokémon
Search	✓	Filters by name as the user types
Search results sorting	✓	Sorting is applied after filtering
2+ sort properties	✓	ID, name, height, weight, base experience
Ascending	✓	Supported
Descending	✓	Supported

Requirement	Status	Why
Gallery media	✓	Official Pokémon artwork
Gallery filtering	✓	Type filters
Multiple filters	✓	<code>Set<string></code> allows multiple selected types
List → Details	✓	Clicking a list row navigates to <code>/pokemon/:id</code>
Gallery → Details	✓	Clicking a gallery card navigates to <code>/pokemon/:id</code>
Details	✓	Image, types, height, weight, abilities, experience, stats
Previous / Next	✓	Uses the loaded Pokémon array and disables at boundaries
React Router	✓	<code>BrowserRouter</code> , <code>Routes</code> , <code>Route</code> , <code>NavLink</code> , <code>useNavigate</code>
Axios	✓	API calls use Axios
TypeScript	✓	Strict TypeScript configuration
Design	✓	CSS modules, responsive layout, polished UI
No inline styling	✓	I don't see inline <code>style={{...}}</code> usage
GitHub Pages	✓	Vite base + Pages workflow + <code>404.html</code> fallback

A few things you did particularly well

1. The sorting is actually better than the minimum requirement

You support:

⌄ Plain text

National No.
Name
Height
Weight
Base Experience

with:

</> Plain text

```
Ascending
Descending
```

So you're comfortably satisfying the "at least 2 properties" requirement.

2. Your gallery supports multiple filters

This is especially good:

</> TypeScript

```
const [selectedTypes, setSelectedTypes] = useState<Set<string>>(new Set());
```

and:

</> TypeScript

```
p.types.some((t) => selectedTypes.has(t.type.name))
```

That means a user can select multiple types rather than being limited to one.

3. Your API caching is thoughtful

You have:

</> TypeScript

```
const CACHE_KEY = 'mp2-pokemon-cache-v1';
```

and use `sessionStorage` .

That's a nice response to the README's warning about API rate limiting. The app avoids repeatedly downloading all 151 Pokémon during the same browser session.

4. Your Previous/Next implementation is solid

This is particularly clean:

</> TypeScript

```
const currentIndex = useMemo(
  () => pokemon.findIndex((p) => p.id === Number(id)),
```

```
[pokemon, id],  
);
```

followed by:

</> TypeScript

```
const hasPrev = currentIndex > 0;  
const hasNext = currentIndex < pokemon.length - 1;
```

So #1 correctly has no Previous and #151 correctly has no Next.

5. GitHub Pages routing was handled well

This is important.

You have:

</> TypeScript

```
<BrowserRouter basename={import.meta.env.BASE_URL}>
```

and:

</> TypeScript

```
base: '/mp2/',
```

Your `package.json` also creates:

</> Plain text

```
dist/404.html
```

from `index.html` during the production build:

</> JSON

```
"build": "tsc -b && vite build && node -e \"require('fs').copyFileSync('dist/index.html','dist/404.html')\""
```

That's a smart solution for React Router on GitHub Pages.

One thing I would fix before submitting

There's one **academic/submission requirement** I would pay attention to.

The assignment says:

All sources (code as well as reading material) that you reference to complete the assignment must be declared in the submission.

Your code clearly uses external technologies/resources including:

`</>` Plain text

PokéAPI
Axios
React Router
React
TypeScript
Vite

The README also explicitly says that if LLMs were used, you must submit the chat logs and answer the LLM survey questions.

Since you used ChatGPT throughout this development, make absolutely sure you submit the chat log as required.

Don't hide the LLM usage. The course explicitly permits it **provided you disclose it**.

One small technical improvement

Your clickable list rows and gallery cards use:

`</>` TypeScript

```
<div
  onClick={...}
  role="button"
  tabIndex={0}
>
```

You handle `Enter` , which is good:

`</>` TypeScript

```
onKeyDown={(e) => {  
  if (e.key === 'Enter') ...  
}}
```

but you don't handle the **Space** key.

This isn't likely to affect the rubric, but for accessibility I'd eventually use a `<Link>` instead of making a `<div>` behave like a button.

For example, a list item could simply be:

`</>` TypeScript

```
<Link to={` /pokemon/${pokemon.id}`}>
```

This automatically gives you proper keyboard navigation and semantics.

I would call this optional, though. I wouldn't risk a large refactor immediately before submission.

Your project structure is also good

You ended up with a much better structure than the initial version:

`</>` Plain text

```
src/  
├─ api/  
│   └─ pokeapi.ts  
├─ components/  
│   ├─ LoadingSpinner.tsx  
│   ├─ Navbar.tsx  
│   ├─ PokemonGalleryCard.tsx  
│   ├─ PokemonListRow.tsx  
│   ├─ SearchBar.tsx  
│   ├─ SortControls.tsx  
│   ├─ TypeBadge.tsx  
│   └─ TypeFilter.tsx  
├─ context/  
│   └─ PokemonContext.tsx  
├─ pages/  
│   ├─ ListPage.tsx  
│   ├─ GalleryPage.tsx  
│   └─ DetailPage.tsx  
├─ types/  
│   └─ pokemon.ts
```

That's a very reasonable React architecture for this assignment.

My verdict

Functionally, yes — your final codebase is in good shape for the MP. I would not rewrite the application at this point.

The main things I'd do before submitting are:

1. Run `npm run lint`
2. Run `npm run build`
3. Test the deployed URL:
 - `/mp2/`
 - `/mp2/gallery`
 - `/mp2/pokemon/25`
 - `refresh /mp2/pokemon/25`
4. Verify Previous/Next at #1 and #151.
5. Make sure your **GitHub repo is public**.
6. Submit the **ChatGPT conversation/log as required by the course**.
7. Prepare the **3-minute demo** showing each rubric item.

If `npm run lint` is clean and the deployed detail URL survives a refresh, **I would stop making functional changes** and move on to the demo/submission preparation. </>